



Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business[Go from Data to Strategy: Tepper School of Business](#)

Training a PyTorch Model with DataLoader and Dataset

by [Adrian Tam](#) on [April 8, 2023](#) in [Deep Learning with PyTorch](#) 5

Share

Share

Post

When you build and train a PyTorch deep learning model, you can provide the training data in several different ways. Ultimately, a PyTorch model works like a function that takes a PyTorch tensor and returns you another tensor. You have a lot of freedom in how to get the input tensors. Probably the easiest is to prepare a large tensor of the entire dataset and extract a small batch from it in each training step. But you will see that using the DataLoader can save you a few lines of code in dealing with data.

In this post, you will see how you can use the the Data and DataLoader in PyTorch. After finishing this post, you will learn:

- How to create and use DataLoader to train your PyTorch model
- How to use Data class to generate data on the fly

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Training a PyTorch Model with DataLoader and Dataset
Photo by Emmanuel Appiah. Some rights reserved.

Overview

This post is divided into three parts; they are:

- What is DataLoader?
- Using DataLoader in a Training Loop

What is `DataLoader`?

To train a deep learning model, you need data. Usually data is available as a dataset. In a dataset, there are a lot of data sample or instances. You can ask the model to take one sample at a time but usually you would let the model to process one batch of several samples. You may create a batch by extracting a slice from the dataset, using the slicing syntax on the tensor. For a better quality of training, you may also want to shuffle the entire dataset on each epoch so no two batch would be the same in the entire training loop. Sometimes, you may introduce **data augmentation** to manually introduce more variance to the data. This is common for image-related tasks, which you can randomly tilt or zoom the image a bit to generate a lot of data sample from a few images.

You can imagine there can be a lot of code to write to do all these. But it is much easier with the `DataLoader`.

The following is an example of how create a `DataLoader` and take a batch from it. In this example, the `sonar dataset` is used and ultimately, it is converted into PyTorch tensors and passed on to `DataLoader`:

```
1 import pandas as pd
2 import torch
3 from torch.utils.data import Dataset, DataLoader
4 from sklearn.preprocessing import LabelEncoder
5
6 # Read data, convert to NumPy arrays
7 data = pd.read_csv("sonar.csv", header=None)
8 X = data.iloc[:, 0:60].values
9 y = data.iloc[:, 60].values
10
11 # encode class values as integers
12 encoder = LabelEncoder()
13 encoder.fit(y)
14 y = encoder.transform(y)
15
16 # convert into PyTorch tensors
17 X = torch.tensor(X, dtype=torch.float32)
18 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
19
20 # create DataLoader, then take one batch
21 loader = DataLoader(list(zip(X,y)), shuffle=True, batch_size=16)
22 for X_batch, y_batch in loader:
23     print(X_batch, y_batch)
24     break
```

You can see from the output of above that `X_batch` and `y_batch` are PyTorch tensors. The loader is an instance of `DataLoader` class which can work like an iterable. Each time you read from it, you get a batch of features and targets from the original dataset.

When you create a `DataLoader` instance, you need to provide a list of sample pairs. Each sample pair is one data sample of feature and the corresponding target. A list is required because `DataLoader` expect to use `len()` to find the total size of the dataset and using array index to retrieve a particular sample. The batch size is a parameter to `DataLoader` so it knows how to create a batch from the entire dataset. You should almost always use `shuffle=True` so every time you load the data, the samples are shuffled. It is useful for training because in each epoch, you are going to read every batch once. When you proceed from one epoch to another, as `DataLoader` knows you depleted all the batches, it will re-shuffle so you get a new combination of samples.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Using `DataLoader` in a Training Loop

The following is an example to make use of `DataLoader` in a training loop:

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn.model_selection import train_test_split
5
6 # train-test split for evaluation of the model
7 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
```

```

8
9 # set up DataLoader for training set
10 loader = DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=16)
11
12 # create model
13 model = nn.Sequential(
14     nn.Linear(60, 60),
15     nn.ReLU(),
16     nn.Linear(60, 30),
17     nn.ReLU(),
18     nn.Linear(30, 1),
19     nn.Sigmoid()
20 )
21
22 # Train the model
23 n_epochs = 200
24 loss_fn = nn.BCELoss()
25 optimizer = optim.SGD(model.parameters(), lr=0.1)
26 model.train()
27 for epoch in range(n_epochs):
28     for X_batch, y_batch in loader:
29         y_pred = model(X_batch)
30         loss = loss_fn(y_pred, y_batch)
31         optimizer.zero_grad()
32         loss.backward()
33         optimizer.step()
34
35 # evaluate accuracy after training
36 model.eval()
37 y_pred = model(X_test)
38 acc = (y_pred.round() == y_test).float().mean()
39 acc = float(acc)
40 print("Model accuracy: %.2f%%" % (acc*100))

```

You can see that once you created the DataLoader instance, the training loop can only be easier. In the above, only the training set is packaged with a DataLoader because you need to loop through it in batches. You can also create a DataLoader for the test set and use it for model evaluation, but since the accuracy is computed over the entire test set rather than in a batch, the benefit of DataLoader is not significant.

Putting everything together, below is the complete code.

```

1 import pandas as pd
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from torch.utils.data import DataLoader
6 from sklearn.preprocessing import LabelEncoder
7 from sklearn.model_selection import train_test_split
8
9 # Read data, convert to NumPy arrays
10 data = pd.read_csv("sonar.csv", header=None)
11 X = data.iloc[:, 0:60].values
12 y = data.iloc[:, 60].values
13
14 # encode class values as integers
15 encoder = LabelEncoder()
16 encoder.fit(y)
17 y = encoder.transform(y)
18
19 # convert into PyTorch tensors
20 X = torch.tensor(X, dtype=torch.float32)
21 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
22
23 # train-test split for evaluation of the model
24 X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
25
26 # set up DataLoader for training set
27 loader = DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=16)
28
29 # create model
30 model = nn.Sequential(
31     nn.Linear(60, 60),
32     nn.ReLU(),
33     nn.Linear(60, 30),
34     nn.ReLU(),
35     nn.Linear(30, 1),
36     nn.Sigmoid()
37 )
38
39 # Train the model
40 n_epochs = 200
41 loss_fn = nn.BCELoss()
42 optimizer = optim.SGD(model.parameters(), lr=0.1)
43 model.train()
44 for epoch in range(n_epochs):
45     for X_batch, y_batch in loader:
46         y_pred = model(X_batch)
47         loss = loss_fn(y_pred, y_batch)
48         optimizer.zero_grad()
49         loss.backward()
50         optimizer.step()
51
52 # evaluate accuracy after training
53 model.eval()
54 y_pred = model(X_test)
55 acc = (y_pred.round() == y_test).float().mean()

```

```
56 acc = float(acc)
57 print("Model accuracy: %.2f%%" % (acc*100))
```

Create Data Iterator using Dataset Class

In PyTorch, there is a Dataset class that can be tightly coupled with the DataLoader class. Recall that DataLoader expects its first argument can work with len() and with array index. The Dataset class is a base class for this. The reason you may want to use Dataset class is there are some special handling before you can get the data sample. For example, data should be read from database or disk and you only want to keep a few samples in memory rather than prefetch everything. Another example is to perform real-time preprocessing of data, such as random augmentation that is common in image tasks.

To use Dataset class, you just subclass from it and implement two member functions. Below is an example:

```
1 from torch.utils.data import Dataset
2
3 class SonarDataset(Dataset):
4     def __init__(self, X, y):
5         # convert into PyTorch tensors and remember them
6         self.X = torch.tensor(X, dtype=torch.float32)
7         self.y = torch.tensor(y, dtype=torch.float32)
8
9     def __len__(self):
10        # this should return the size of the dataset
11        return len(self.X)
12
13    def __getitem__(self, idx):
14        # this should return one sample from the dataset
15        features = self.X[idx]
16        target = self.y[idx]
17        return features, target
```

This is not the most powerful way to use Dataset but simple enough to demonstrate how it works. With this, you can create a DataLoader and use it for model training. Modifying from the previous example, you have the following:

```
1 ...
2
3 # set up DataLoader for training set
4 dataset = SonarDataset(X_train, y_train)
5 loader = DataLoader(dataset, shuffle=True, batch_size=16)
6
7 # create model
8 model = nn.Sequential(
9     nn.Linear(60, 60),
10    nn.ReLU(),
11    nn.Linear(60, 30),
12    nn.ReLU(),
13    nn.Linear(30, 1),
14    nn.Sigmoid()
15 )
16
17 # Train the model
18 n_epochs = 200
19 loss_fn = nn.BCELoss()
20 optimizer = optim.SGD(model.parameters(), lr=0.1)
21 model.train()
22 for epoch in range(n_epochs):
23     for X_batch, y_batch in loader:
24         y_pred = model(X_batch)
25         loss = loss_fn(y_pred, y_batch)
26         optimizer.zero_grad()
27         loss.backward()
28         optimizer.step()
29
30 # evaluate accuracy after training
31 model.eval()
32 y_pred = model(torch.tensor(X_test, dtype=torch.float32))
33 y_test = torch.tensor(y_test, dtype=torch.float32)
34 acc = (y_pred.round() == y_test).float().mean()
35 acc = float(acc)
36 print("Model accuracy: %.2f%%" % (acc*100))
```

You set up dataset as an instance of SonarDataset which you implemented the __len__() and __getitem__() functions. This is used in place of the list in the previous example to set up the DataLoader instance. Afterward, everything is the same in the training loop. Note that you still use PyTorch tensors directly for the test set in the example.

In the __getitem__() function, you take an integer that works like an array index and returns a pair, the features and the target. You can implement anything in this function: Run some code to generate a synthetic data sample, read data on the fly from the internet, or add random variations to the data. You will also find it useful in the situation that you cannot keep the entire dataset in memory, so you can load only the data samples that you need it.

In fact, since you created a PyTorch dataset, you don't need to use scikit-learn to split data into training set and test set. In torch.utils.data submodule, you have a function random_split() that works with Dataset class for the same purpose. A full example is below:

```
1 import pandas as pd
2 import torch
```

```

3 import torch.nn as nn
4 import torch.optim as optim
5 from torch.utils.data import Dataset, DataLoader, random_split, default_collate
6 from sklearn.preprocessing import LabelEncoder
7
8 # Read data, convert to NumPy arrays
9 data = pd.read_csv("sonar.csv", header=None)
10 X = data.iloc[:, 0:60].values
11 y = data.iloc[:, 60].values
12
13 # encode class values as integers
14 encoder = LabelEncoder()
15 encoder.fit(y)
16 y = encoder.transform(y).reshape(-1, 1)
17
18 class SonarDataset(Dataset):
19     def __init__(self, X, y):
20         # convert into PyTorch tensors and remember them
21         self.X = torch.tensor(X, dtype=torch.float32)
22         self.y = torch.tensor(y, dtype=torch.float32)
23
24     def __len__(self):
25         # this should return the size of the dataset
26         return len(self.X)
27
28     def __getitem__(self, idx):
29         # this should return one sample from the dataset
30         features = self.X[idx]
31         target = self.y[idx]
32         return features, target
33
34 # set up DataLoader for data set
35 dataset = SonarDataset(X, y)
36 trainset, testset = random_split(dataset, [0.7, 0.3])
37 loader = DataLoader(trainset, shuffle=True, batch_size=16)
38
39 # create model
40 model = nn.Sequential(
41     nn.Linear(60, 60),
42     nn.ReLU(),
43     nn.Linear(60, 30),
44     nn.ReLU(),
45     nn.Linear(30, 1),
46     nn.Sigmoid()
47 )
48
49 # Train the model
50 n_epochs = 200
51 loss_fn = nn.BCELoss()
52 optimizer = optim.SGD(model.parameters(), lr=0.1)
53 model.train()
54 for epoch in range(n_epochs):
55     for X_batch, y_batch in loader:
56         y_pred = model(X_batch)
57         loss = loss_fn(y_pred, y_batch)
58         optimizer.zero_grad()
59         loss.backward()
60         optimizer.step()
61
62 # create one test tensor from the testset
63 X_test, y_test = default_collate(testset)
64 model.eval()
65 y_pred = model(X_test)
66 acc = (y_pred.round() == y_test).float().mean()
67 acc = float(acc)
68 print("Model accuracy: %.2f%%" % (acc*100))

```

It is very similar to the example you have before. Beware that the PyTorch model still needs a tensor as input, not a Dataset. Hence in the above, you need to use the `default_collate()` function to collect samples from a dataset into tensors.

Further Readings

This section provides more resources on the topic if you are looking to go deeper.

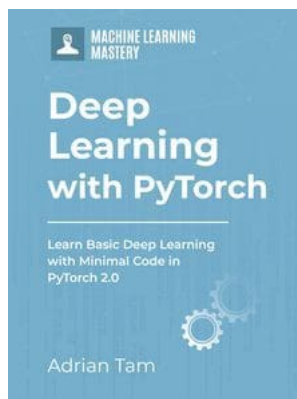
- [torch.utils.data](#) from PyTorch documentation
- [Datasets and DataLoaders](#) from PyTorch tutorial

Summary

In this post, you learned how to use `DataLoader` to create shuffled batches of data and how to use `Dataset` to provide data samples. Specifically you learned:

- `DataLoader` as a convenient way of providing batches of data to the training loop
- How to use `Dataset` to produce data samples
- How combine `Dataset` and `DataLoader` to generate batches of data on the fly for model training

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

Share Share Post

More On This Topic



Mini-Batch Gradient Descent and DataLoader in PyTorch



How to Prepare a Photo Caption Dataset for Training...



Training a Linear Regression Model in PyTorch



Training a Single Output Multilinear Regression...



Training a Multi-Target Multilinear Regression Model...



Converting Pandas DataFrames to PyTorch DataLoaders...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

< Using Learning Rate Schedule in PyTorch Training

Understand Model Behavior During Training by Visualizing Metrics >

5 Responses to *Training a PyTorch Model with DataLoader and Dataset***Aditya** February 26, 2023 at 12:32 am #

REPLY ↩

Hi, is shuffling appropriate for forecasting problems

**Adrian Tam** March 15, 2023 at 5:43 am #

REPLY ↩

Usually no. It sounds like you're talking about a time series problem and we do not want to lose the time ordering information. Therefore, shuffling is not recommended. But you can transform a time series into windows and shuffle the windows. Hope that helps.

**Matthew Avaylon** August 8, 2023 at 2:52 am #

REPLY ↩

In the intro you mentioned that pytorch models can handle taking a large tensor of data. Does that mean I can load in the MNIST dataset, set a `x_train` and `y_train` of all the training data as tensors and train like this?

for epoch in range(20):

```
print("epoch:" + str(epoch))
```

```
model.train() # puts the model in training mode
```

```
y_pred = model(X_train) # Forward pass
```

```
loss_calc = loss_func(y_pred, y_train)
```

```
optimizer.zero_grad()
```

```
loss_calc.backward()
```

```
optimizer.step()
```

I'm a little confused on what dataloader actually does. The loader iterates over the data in batches and will feed the batch to the model. Is that saying that its loading these batches into memory where each batch can be thought of a small tensor of data that is being fed into the model? Much like how we can feed a whole tensor of data in my opening statement, this is feeding a tensor of a batch.

**Peggy** June 13, 2024 at 12:08 pm #

REPLY ↩

For Multi-Instance Learning (MIL), my dataset includes unique IDs, features, and labels for training.

For prediction, I need to provide the dataset with unique IDs and features, but without labels.

Therefore, may I ask should modify my dataset class to handle data without labels during prediction?

Thanks!

**James Carmichael** June 14, 2024 at 6:48 am #

REPLY ↩

Hi Peggy...Yes, you should modify your dataset class to handle data without labels during prediction in Multi-Instance Learning (MIL). Typically, this involves creating a dataset class that can manage both training (with labels) and prediction (without labels) scenarios.

Here's a general approach to modifying your dataset class:

1. Define the Dataset Class

You can create a dataset class that accepts data in both labeled and unlabeled forms. This class should be able to distinguish whether it's being used for training or prediction based on the presence of labels.

2. Handling Different Scenarios

You can add a parameter to indicate whether the dataset includes labels or not. If labels are not provided, the class should handle the data accordingly during prediction.

Example in PyTorch

Here's a basic example in PyTorch to illustrate this:

```
python
import torch
from torch.utils.data import Dataset
```

```

class MILDataset(Dataset):
    def __init__(self, data, labels=None, mode='train'):
        """
        Args:
        data (list or array-like): List of features or instances.
        labels (list or array-like, optional): List of labels corresponding to the data. Default is None.
        mode (str): Either 'train' or 'predict' to indicate the mode of operation. Default is 'train'.
        """

        self.data = data
        self.labels = labels
        self.mode = mode

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        if torch.is_tensor(idx):
            idx = idx.tolist()

        sample = self.data[idx]

        if self.mode == 'train':
            if self.labels is None:
                raise ValueError("Labels must be provided in training mode.")
            label = self.labels[idx]
            return sample, label
        elif self.mode == 'predict':
            return sample
        else:
            raise ValueError("Mode should be either 'train' or 'predict'.")

# Example usage:
# For training
train_data = [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]]
train_labels = [0, 1, 0]
train_dataset = MILDataset(data=train_data, labels=train_labels, mode='train')

# For prediction
predict_data = [[0.7, 0.8], [0.9, 1.0]]
predict_dataset = MILDataset(data=predict_data, mode='predict')

### Explanation
- **Initialization (__init__ method)**:
    - The data parameter holds the features.
    - The labels parameter is optional and only required in training mode.
    - The mode parameter specifies whether the dataset is for training or prediction.

- **Length (__len__ method)**:
    - Returns the number of instances in the dataset.

- **Get Item (__getitem__ method)**:
    - If in training mode ('train'), it returns a tuple of (sample, label).
    - If in prediction mode ('predict'), it returns only the sample (feature vector).

### Using the Dataset with DataLoader
You can use this dataset class with PyTorch's DataLoader for both training and prediction:

python
from torch.utils.data import DataLoader

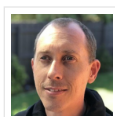
# For training
train_loader = DataLoader(train_dataset, batch_size=2, shuffle=True)

# For prediction
predict_loader = DataLoader(predict_dataset, batch_size=2, shuffle=False)

This structure allows your dataset class to be flexible and handle both training (with labels) and prediction (without labels) scenarios efficiently.

```

Leave a Reply

Name (required) Email (will not be published) (required)[SUBMIT COMMENT](#)**Welcome!**

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:**Picked for you:**

PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The Deep Learning with PyTorch EBook
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved